

AI Interaction Log

CIE-1 – Operating Systems

TY IT

Roll No. 33284

Name: Jayachandra Vandanapu

I used AI during this assignment mainly for understanding Linux commands, planning the locality experiment, debugging the dashboard, and checking my approach. Whenever an answer depended on the current state of my machine, I tested it using actual output from my WSL2 environment.

1. Entry 1 – AI WAS WRONG

My prompt

What will happen when I check the process with the highest CPU usage using `ps`? Should I expect the highest CPU value to be greater than 0.0%?

AI answer

The AI predicted that the process with the highest CPU usage would have a CPU value greater than 0.0%.

How I checked

I ran the command directly on my WSL2 system:

```
ps -eo pid,comm,stat,%cpu,%mem --sort=-%cpu | head -n 6
```

Evidence

PID	COMMAND	STAT	%CPU	%MEM
1	systemd	Ss	0.0	0.3
93	systemd-udevd	Ss	0.0	0.1
289	Relay(290)	S	0.0	0.0
51	systemd-journal	S<s	0.0	0.4
196	unattended-upgr	Ssl	0.0	0.5

Conclusion

The prediction was incorrect. The process list was sorted by CPU usage, but every process shown in this snapshot had a CPU usage of 0.0%. Therefore, the highest CPU usage in this particular snapshot was also 0.0%.

This showed me that current process information cannot be assumed from a general prediction and should be checked on the actual system.

2. Entry 2 – AI GUIDANCE WAS VERIFIED

My prompt

How should I perform a row-major versus column-major memory locality experiment in C?

AI answer

The AI explained that C stores two-dimensional arrays in row-major order. It suggested comparing row-major and column-major traversal and measuring their execution times over multiple runs.

How I checked

I implemented the experiment in C using a 4096×4096 array of `double` values and compiled it using:

```
gcc -O2 locality.c -o locality_c
```

I then ran both traversal methods three times.

Evidence

The Day 7 checkpoint produced:

```
run,row_major_s,column_major_s,ratio
1,0.022200,0.252546,11.38
2,0.020004,0.251715,12.58
3,0.019275,0.254740,13.22
mean,0.020493,0.253000,12.35
```

Conclusion

The general explanation was correct. Row-major traversal was faster in all three runs. The exact times changed between runs, but the overall pattern remained the same.

3. Entry 3 – AI GUIDANCE WAS USED FOR THE DASHBOARD

My prompt

What is a simple way to create the System Detective dashboard without using a large framework?

AI answer

The AI suggested using a simple Node.js server with HTML, CSS and JavaScript. The server could execute Linux commands and provide the results to the frontend.

How I used it

I implemented the dashboard using:

```
dashboard/
  server.js
  public/
    index.html
    style.css
    script.js
```

The dashboard displays CPU information, memory usage, the top five processes and locality results. I also added an option to run the locality experiment from the dashboard.

How I checked it

I started the server in WSL2 using:

```
cd ~/system-detective/dashboard
node server.js
```

I then tested the locality endpoint using:

```
curl -X POST http://localhost:3000/api/locality/run
```

Result

The dashboard successfully displayed the system information and the actual locality benchmark results. The benchmark was executed by the backend rather than using hardcoded values.

4. Entry 4 – AI GUIDANCE WAS VERIFIED AGAIN

My prompt

How can I check whether the locality experiment is reproducible?

AI answer

The AI suggested running the same benchmark multiple times with the same array size, program and compiler settings. It also pointed out that exact execution times can change because the system is running other processes at the same time.

How I checked

I repeated the same benchmark using the same C program, array size, compiler optimization and number of runs.

The latest API execution produced:

```
run,row_major_s,column_major_s,ratio
1,0.020330,0.262567,12.92
2,0.018746,0.246331,13.14
3,0.017239,0.258203,14.98

Mean row-major time: 0.018772 seconds
Mean column-major time: 0.255700 seconds
Mean ratio: 13.62x
```

Conclusion

The exact values were different from the earlier checkpoint, but row-major traversal was still faster than column-major traversal. This confirmed that the timings can vary between executions while the overall locality behaviour remains consistent.

5. What I Take From This

The main thing I learned from using AI in this project is that it is useful for explaining concepts and suggesting approaches, but it does not know the current state of my machine.

The incorrect CPU prediction was the clearest example. The AI gave a reasonable-sounding prediction, but the actual command output showed something different.

For this project, my practical rule was simple: use AI to understand what to test, but use my own Linux commands to find out what is actually happening on my machine.

The system-specific numbers in the project were obtained from the actual WSL2 environment rather than being taken from AI responses.